AGENTIC AI

Degrees of Autonomy, Tool Use, Planning, and Reflection

Professional explanatory notes with enterprise examples, revision summary, and brainstorming questions

CORE IDEA
Agentic behavior is best understood as a spectrum. The more decisions an AI system makes about planning, tool selection, execution, and revision, the more agentic it is.

Prepared for: Saif Ullah
PS Technical Consultant II

Document Guide

This document turns the supplied lecture transcript into a structured learning resource. It preserves the central argument while expanding each concept with practical examples and enterprise implementation considerations.

Section	Focus
1. Foundations	Why “agentic” is a spectrum rather than a binary label
2. Autonomy spectrum	Low, semi, and highly autonomous systems
3. Core capabilities	Planning, decisions, tools, actions, reflection, and iteration
4. Research-agent walkthrough	Black-hole essay example at different autonomy levels
5. Enterprise application	Banking and production-support patterns
6. Design and governance	Control boundaries, guardrails, validation, and observability
7. Revision summary	Condensed definitions and memory hooks
8. Brainstorming questions	Concept checks, design exercises, and interview prompts

HOW TO USE THESE NOTES

First read Sections 1 to 4 for the conceptual model. Then use Sections 5 and 6 to connect the model to enterprise systems. Finish with the revision summary and answer the brainstorming questions without looking back.

1. Foundations of Agentic AI

1.1 The definition problem

The AI community has often debated whether a particular application is a “true agent.” A system may be called an agent by its builder, while another practitioner may reject that label because the system is not autonomous enough. This creates a binary classification problem: the application is either an agent or it is not.

The term “agentic” avoids this unproductive boundary debate. Because it is an adjective, it describes the degree to which a system exhibits agent-like behavior. The useful engineering question therefore changes from “Is it an agent?” to “Which decisions does the system make, which decisions remain with the programmer, and how much independent action is permitted?”

DEFINITION
Agentic AI refers to AI systems or workflows that pursue a goal through some combination of reasoning, planning, tool use, action, evaluation, and iteration. These capabilities may be present to different degrees.

1.2 Binary classification versus a spectrum

Binary view	Spectrum view
Agent or not an agent	Less agentic to more agentic
Focuses on labels	Focuses on system behavior
Encourages definition disputes	Encourages design decisions
Hides differences between systems	Makes autonomy boundaries explicit
Offers little operational guidance	Supports risk-based architecture choices

1.3 The central engineering question

A system becomes more agentic as control shifts from predetermined application logic toward model-directed decisions. The shift can occur in several areas: deciding the next step, selecting a tool, choosing how many sources to inspect, determining whether additional evidence is required, reviewing a draft, retrying after failure, or generating a new function.

MEMORY HOOK
More agentic = more consequential decisions are made by the AI rather than being fixed in advance by the programmer.

2. Degrees of Autonomy

Autonomy is not a single switch. It is a collection of delegated decisions. Two applications may use the same language model and the same tools but still differ significantly in autonomy because one follows a fixed orchestration while the other chooses its own path.

2.1 Low-autonomy agentic workflow

In a low-autonomy workflow, the engineer predetermines the sequence. The LLM may generate search queries, classifications, summaries, or final text, but the program controls the transitions and tool calls.

EXAMPLE: FIXED RESEARCH PIPELINE
User request -> LLM generates three search queries -> application calls web search -> application fetches the top three pages -> LLM writes the essay. The order, number of searches, and number of pages are fixed.

Strengths

- Predictable execution path and bounded cost.
- Simpler testing, debugging, logging, and audit review.
- Clear security boundaries because callable tools and parameters are known.
- Suitable for repeatable enterprise processes with stable requirements.

Limitations

- Cannot adapt well when the task requires an unexpected step.
- May fetch too little or too much information because limits are static.
- Workflow changes usually require code or configuration changes.

2.2 Semi-autonomous agent

A semi-autonomous agent receives controlled discretion. The engineer provides approved tools, policies, limits, and termination conditions, while the model chooses among those options. This is often the preferred enterprise pattern because it combines adaptability with governance.

EXAMPLE: CONTROLLED RESEARCH AGENT
The model may choose web search, recent-news search, or paper search. It may choose how many approved results to open and whether a PDF-to-text tool is required. It cannot access unapproved systems, exceed configured budgets, or publish without validation.

Typical decision rights

- Select one tool from an approved catalog.
- Create a small plan and revise it after receiving tool results.
- Decide whether evidence is sufficient within a maximum iteration limit.
- Escalate to a human when confidence or authorization is insufficient.

2.3 Highly autonomous agent

At the high-autonomy end, the system determines much of the execution sequence. It can decompose a goal into tasks, select and chain tools, respond to new findings, create code or helper functions, evaluate progress, and continue until a stopping condition is reached.

IMPORTANT
High autonomy is not automatically better. It offers flexibility for novel work, but increases unpredictability, security exposure, evaluation difficulty, latency, and cost.

Dimension	Low autonomy	Semi-autonomous	High autonomy
Sequence	Predetermined	Partly model-selected	Dynamically model-generated
Tool choice	Hard-coded	Chosen from approved tools	Broad or extensible
Iterations	Fixed	Bounded and conditional	Adaptive
Predictability	High	Moderate	Lower
Control model	Application-led	Shared	Agent-led
Best fit	Stable repetitive work	Enterprise adaptive work	Open-ended research and experimentation

3. Core Capabilities That Increase Agentic Behavior

3.1 Goal interpretation

The system converts a user request into an operational objective. A weak interpretation simply restates the prompt. A stronger interpretation identifies the expected output, quality criteria, constraints, and missing dependencies.

EXAMPLE
“Analyze this ATM failure” can be interpreted as: identify the most likely failure category, cite supporting log evidence, recommend safe remediation, state uncertainty, and produce a support-ready incident summary.

3.2 Planning and task decomposition

Planning means selecting intermediate steps needed to reach the objective. Task decomposition breaks a broad goal into manageable subtasks. In fixed workflows, the programmer supplies the plan. In more agentic workflows, the model creates or adjusts it.

1. Understand requested outcome and constraints.
2. Identify information sources and tools.
3. Order tasks based on dependencies.
4. Execute a step and inspect the result.
5. Revise the plan if the evidence changes.
6. Stop when acceptance criteria are satisfied or escalation is required.

3.3 Tool selection and tool use

A tool is non-LLM software used to perform an action or retrieve information. Examples include search APIs, databases, file parsers, calculators, code execution, ticketing systems, and enterprise knowledge retrieval. Tools connect language-model reasoning to external data and operations.

Tool category	Purpose	Enterprise example
Retrieval	Obtain relevant evidence	Search knowledge base or incident history
Parsing	Convert content into usable text	Extract text from PDF or logs
Computation	Perform deterministic processing	Calculate trends or parse timestamps
Action	Change an external system	Create a ticket or restart an approved service
Communication	Deliver or request information	Draft an incident update for review
Validation	Check output against rules	Schema validation and policy checks

3.4 Action and execution

Generating a recommendation is different from executing it. Once an agent can change an external system, the impact and risk increase. Read-only retrieval is usually safer than write operations; reversible changes are safer than destructive actions; and scoped actions are safer than unrestricted access.

3.5 Reflection and self-evaluation

Reflection is a deliberate review step in which the system evaluates an intermediate output against explicit criteria. It may check completeness, evidence quality, contradictions, policy compliance, formatting, or whether a second retrieval cycle is justified.

REFLECTION LOOP
Draft -> evaluate against criteria -> identify gaps -> retrieve or reason again -> revise -> validate -> final response.

Reflection should not be treated as proof of correctness. A model reviewing its own output can still preserve the same mistaken assumption. Strong implementations combine reflection with independent evidence, deterministic validation, or a separate evaluator.

3.6 Memory and state

State helps an agent retain the task objective, completed steps, tool outputs, decisions, and remaining work. Short-term state supports one execution. Longer-term memory can preserve preferences or prior outcomes, but introduces privacy, freshness, and incorrect-memory risks.

3.7 Stopping conditions

An agent needs explicit termination logic. Without it, the system may loop, over-search, or repeatedly revise an already acceptable result. Useful stopping conditions include acceptance criteria met, maximum iterations reached, budget exhausted, required evidence unavailable, low confidence, or human approval required.

4. Research-Agent Walkthrough: Essay on Black Holes

4.1 Less autonomous design

FLOW
Input -> generate search terms -> web search -> fetch fixed number of pages -> write essay -> output.

This approach can work well when the output format and research depth are consistent. The model controls the content of search terms and prose, while orchestration remains deterministic.

4.2 More autonomous design

FLOW
Input -> plan -> choose source type -> search -> inspect results -> decide what to fetch -> convert documents if required -> synthesize -> reflect -> search again if needed -> finalize.

The model now makes decisions that were previously coded by the engineer. It may determine that recent research papers are more appropriate than general web pages, that a PDF must be converted to text, or that the first evidence set is insufficient.

4.3 Decision-by-decision comparison

Decision	Less autonomous	More autonomous
Source type	Web search fixed by engineer	Model chooses web, news, or papers
Query count	Fixed	Selected according to task
Pages fetched	Fixed top N	Chosen based on relevance
PDF handling	Predefined parser step	Invoked only when required
Revision	None or fixed once	Conditional reflection loop
Additional research	Not available	Triggered by evidence gaps
Finalization	After fixed pipeline	After criteria or limits are reached

4.4 Why the exact path can change

Dynamic execution is useful because research tasks are not uniform. One topic may be answered from a few reliable pages; another may require academic papers, document conversion, contradiction resolution, or additional searches. The same flexibility that improves coverage also makes behavior harder to predict and test.

5. Enterprise Application Patterns

5.1 Recommended pattern for regulated environments

For banking, production support, endpoint management, and infrastructure automation, semi-autonomous design is usually the practical target. The model can interpret evidence and recommend or select bounded next steps, while the application enforces identity, permissions, policy, approval, and deterministic validation.

5.2 Example: ATM incident-analysis assistant

OBJECTIVE
Reduce analysis time without allowing the AI to make uncontrolled production changes.

7. Ingest ticket details and approved diagnostic logs.
8. Classify the issue using known error signatures and contextual evidence.
9. Retrieve related knowledge articles and previous resolutions.
10. Build a cited problem statement and probable cause.
11. Recommend remediation based on permitted runbooks.
12. Request human approval before any production action.
13. Generate a customer-facing update and internal technical record.

Safe autonomy boundaries

Agent may do	Approval required	Agent must not do
Read approved logs and knowledge	Restart a production service	Use credentials outside its identity
Suggest likely classification	Modify application configuration	Execute arbitrary generated code
Draft an RCA and update	Deploy a package	Hide low confidence or missing evidence
Compare evidence with runbooks	Close a customer incident	Bypass change-management controls

5.3 Example: SCCM or Intune deployment assistant

A controlled deployment assistant can validate prerequisites, inspect deployment status, correlate common failure codes, draft remediation commands, and prepare a rollout summary. Autonomy should be reduced for broad device actions, policy modification, tenant-wide assignments, or destructive remediation.

5.4 Example: enterprise research assistant

An enterprise research assistant can search approved documents, retrieve relevant passages, compare sources, produce a structured brief, and flag unresolved conflicts. Its output should identify sources and distinguish retrieved facts from recommendations.

5.5 Autonomy should follow impact

RISK RULE
The greater the potential impact, irreversibility, scope, or data sensitivity of an action, the stronger the control boundary should be.

Action type	Suggested control
Read public or approved internal content	Automatic with logging
Generate analysis or draft communication	Automatic, followed by user review where appropriate
Create a reversible low-impact record	Policy-based permission and full audit trail
Modify production configuration	Explicit authorization and pre/post validation
Perform destructive or high-scope operation	Strong human approval or prohibit autonomous execution

6. Architecture, Governance, and Production Controls

6.1 A practical controlled-agent architecture

- User/channel layer: captures the request and identity context.
- Orchestrator: maintains state, iteration limits, and workflow policy.
- LLM reasoning layer: interprets the goal, proposes plans, and selects permitted tools.
- Tool gateway: exposes only allow-listed operations with typed inputs.
- Retrieval layer: provides grounded enterprise evidence.
- Policy and authorization layer: checks identity, scope, data classification, and action impact.
- Validation layer: verifies schemas, citations, business rules, and output constraints.
- Approval layer: pauses consequential actions for authorized review.
- Observability layer: records prompts, plans, calls, results, failures, cost, and decisions.

6.2 Guardrails

Guardrail	Purpose
Least privilege	Limit the agent to necessary data and operations
Allow-listed tools	Prevent unrestricted capability expansion
Typed schemas	Reject malformed tool requests and outputs
Input validation	Reduce injection and unsafe parameter risks
Iteration and cost limits	Prevent runaway loops and resource use
Human approval gates	Control consequential or irreversible actions

Guardrail	Purpose
Source grounding	Tie claims to retrieved evidence
Audit logging	Support troubleshooting, compliance, and review
Fail-safe behavior	Stop or escalate when uncertain

6.3 Testing strategy

- Unit-test deterministic tools independently from the LLM.
- Use scenario suites for normal, incomplete, conflicting, malicious, and out-of-scope inputs.
- Evaluate task completion, evidence quality, policy compliance, tool correctness, latency, and cost.
- Test authorization boundaries and confirm the agent cannot access disallowed tools or scopes.
- Replay production failures in a safe environment and add them to regression tests.
- Use canary rollout and monitor outcomes before expanding permissions or user scope.

6.4 Common failure modes

Failure mode	What it looks like	Mitigation
Poor planning	Steps are missing or badly ordered	Plan validation and bounded templates
Wrong tool selection	Irrelevant or unsafe tool is chosen	Tool descriptions, allow-lists, policy checks
Hallucinated evidence	Claims are not supported by sources	Grounding and citation validation
Looping	Repeated searches or revisions	Iteration, time, and budget limits
Overconfident action	Acts despite weak evidence	Confidence thresholds and approval gates
Prompt injection	Retrieved text attempts to control the agent	Treat external content as data; enforce policy outside the model
Stale memory	Old context influences a current task	Freshness rules and user-visible memory controls

7. Consolidated Revision Summary

Concept	Concise definition
Agentic	Describes the degree of agent-like behavior in a system.
Autonomy	The amount of independent decision-making delegated to the system.
Low autonomy	Sequence and tools are primarily predetermined by the engineer.
Semi-autonomous	The model makes bounded decisions using predefined tools and policies.
High autonomy	The model dynamically plans, selects actions, and adapts execution.
Tool use	Calling external software to retrieve information or perform an action.
Planning	Selecting and ordering intermediate steps toward a goal.
Reflection	Evaluating and revising an intermediate result against criteria.
State	Task context, completed work, evidence, and outstanding steps.
Guardrail	A technical or procedural control that constrains behavior.
Human in the loop	A person reviews or authorizes selected decisions or actions.

Seven key takeaways

14. Agentic AI is a spectrum, not a binary category.
15. Autonomy depends on which decisions are delegated, not merely on whether an LLM is present.
16. Fixed workflows remain valuable and are often easier to control.
17. Tool use extends an LLM from text generation into retrieval and action.
18. Reflection can improve quality but does not guarantee correctness.
19. Highly autonomous systems are flexible but less predictable and harder to govern.
20. The best enterprise design maximizes useful autonomy within explicit safety and operational boundaries.

ONE-LINE SUMMARY
Do not maximize autonomy for its own sake. Assign the minimum autonomy needed to deliver the required business value safely and reliably.

8. Brainstorming, Review, and Interview Questions

8.1 Concept checks

1. Why is “agentic” more useful than arguing whether a system is a true agent?
2. Which decisions can move a system from low autonomy to semi-autonomy?
3. Why can two systems using the same LLM have different autonomy levels?
4. How is an LLM call different from a tool call?
5. Why does reflection not guarantee correctness?
6. What is the relationship between dynamic planning and unpredictability?
7. Why are stopping conditions essential in an agentic loop?
8. When is a fixed deterministic workflow preferable to an adaptive one?

8.2 Architecture questions

1. Design a .NET-based agent orchestrator. Which components should remain deterministic?
2. How would you expose enterprise tools without allowing arbitrary function execution?
3. What information should be logged for each tool decision and result?
4. How would you validate model-generated JSON before executing a tool?
5. Where would you place authorization checks: prompt, orchestrator, tool, or all three?
6. How would you prevent retrieved documents from overriding system policy?
7. What conditions should cause an agent to stop and escalate to a human?

8.3 Enterprise scenario exercises

Scenario	Design challenge
ATM support	An agent identifies a service failure and recommends a restart. Define what it may read, what it may propose, and what requires approval.
Intune remediation	An agent finds 1,000 devices with a compliance issue. Design a staged, reversible response with scope controls.
Incident RCA	Multiple logs suggest conflicting causes. Define how the agent should represent uncertainty and request more evidence.
Research assistant	The first sources disagree. Define a bounded strategy for further research and final reporting.
Email automation	The agent drafts a customer update based on technical findings. Define review and evidence requirements before sending.

8.4 Advanced discussion prompts

- Is autonomy a property of the model, the workflow, or the full socio-technical system? Defend your answer.
- Can a highly deterministic workflow still be meaningfully agentic? Explain with an example.
- How should organizations measure the business value of additional autonomy?
- What new risks appear when an agent is allowed to write and execute new tools?
- How can an evaluation distinguish genuine task success from a plausible-looking answer?
- What autonomy level would you recommend for a production banking support agent, and why?

9. Practical Design Checklist

Area	Question to answer before production
Goal	Is the expected outcome measurable and unambiguous?
Autonomy	Which decisions are delegated, and why?
Tools	Are tools allow-listed, scoped, typed, and documented?
Identity	Does every action use an authenticated, least-privilege identity?
Data	Are classification, privacy, and retention requirements enforced?
Evidence	Must factual claims be grounded in retrieved sources?
Validation	Which checks are deterministic?
Approval	Which actions require human authorization?
Limits	Are iteration, time, cost, and scope bounded?
Recovery	Can actions be rolled back or safely retried?
Observability	Can reviewers reconstruct what the agent decided and why?
Evaluation	Are success, safety, quality, latency, and cost measured?

FINAL PRINCIPLE
Build the least autonomous system that reliably solves the problem, then add autonomy only where measured benefits justify the additional risk and complexity.